C语言链表详细介绍

一、链表概述

链表（Linked List）是一种基础且重要的线性数据结构，通过节点间的指针关联实现动态存储。与数组的连续内存分配不同，链表的每个节点（Node）包含两部分：数据域（存储元素值）和指针域（指向下一个节点的地址）。这种离散存储特性使得链表在内存使用上更加灵活，无需预先分配固定空间。

1.1 链表与数组的对比

特性	数组	链表
内存分配	静态分配，大小固定	动态分配，大小可变
随机访问	O(1)	O(n)
插入删除	O (n)（需要移动大量元素）	O (1)（已知位置时）
内存效率	高，连续存储	低，需要额外指针空间
缓存性能	好，局部性原理	差，节点分散存储
实现复杂度	简单	复杂

1.2 链表的核心特点

- **动态性**：节点可在程序运行时动态生成和删除，无需预先确定内存大小。
- **非连续性**：节点在内存中的物理位置不一定相邻，通过指针连接形成逻辑上的线性序列。
- **增删高效**：在已知位置的情况下，插入和删除操作仅需修改指针指向，无需移动大量数据。
- **随机访问低效**：访问特定元素需要从头节点开始遍历，无法直接通过索引访问。

二、链表的分类

2.1 按方向性分类

（1）单链表

每个节点仅包含一个指针，指向它的后继节点。这种结构简单，内存开销较小，但只能单向遍历。
节点结构示例：

Code block

```
typedef struct SListNode {
    int data;            // 数据域
    struct SListNode* next;  // 指针域，指向下一个节点
} SListNode;
```

（2）双向链表

每个节点包含两个指针，分别指向前驱节点和后继节点。这种结构支持双向遍历，操作更灵活，但每个节点需要额外的内存空间存储前驱指针。
节点结构示例：

Code block

```
typedef struct DListNode {
    int data;            // 数据域
    struct DListNode* prev;  // 指向前驱节点
    struct DListNode* next;  // 指向后继节点
} DListNode;
```

2.2 按循环性分类

（1）非循环链表

尾节点的指针为空（NULL），表示链表结束。这是最常见的链表形式，结构直观，操作简单。

（2）循环链表

尾节点的指针指向头节点（在带头节点的情况下）或第一个数据节点（在不带头节点的情况下），形成一个闭环结构。这种设计使得从任意节点出发都能遍历整个链表，适合环形调度（如操作系统进程轮转、约瑟夫问题）场景。
单向循环链表节点结构示例：

Code block

```
typedef struct CListNode {
    int data;
    struct CListNode* next;
} CListNode;
```

2.3 按头节点设计分类

（1）带头节点

链表包含一个特殊的头节点（哨兵节点），该节点不存储有效数据，仅用于简化操作逻辑。头节点的存在使得空链表和非空链表的操作可以统一处理，避免了许多边界条件的判断。

（2）不带头节点

链表的第一个节点直接存储有效数据。这种设计更节省空间，但在处理空链表和插入删除头节点时需要特殊处理。

三、单链表的实现与操作

3.1 基本操作

（1）创建节点

Code block

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>

// 创建单链表新节点
SListNode* BuySListNode(int x) {
    SListNode* newnode = (SListNode*)malloc(sizeof(SListNode));
    if (newnode == NULL) {
        perror("malloc failed");
        exit(-1);
    }
    newnode->data = x;
    newnode->next = NULL;
    return newnode;
}
```

（2）头插操作

在链表头部插入新节点：

Code block

```
// 在单链表头部插入新节点
void SLTPushFront(SListNode** pphead, int x) {
    assert(pphead);
    SListNode* newnode = BuySListNode(x);
    newnode->next = *pphead;
    *pphead = newnode;
}
```

（3）尾插操作

在链表尾部插入新节点：

Code block

```
// 在单链表尾部插入新节点
void SLTPushBack(SListNode** pphead, int x) {
    assert(pphead);
    SListNode* newnode = BuySListNode(x);
    if (*pphead == NULL) {
        *pphead = newnode;
    } else {
        SListNode* tail = *pphead;
        while (tail->next != NULL) {
            tail = tail->next;
        }
        tail->next = newnode;
    }
}
```

（4）删除头节点

Code block

```
// 删除单链表头节点
void SLTPopFront(SListNode** pphead) {
    assert(pphead);
    assert(*pphead != NULL);
    SListNode* temp = *pphead;
    *pphead = (*pphead)->next;
    free(temp);
}
```

（5）删除尾节点

Code block

```
// 删除单链表尾节点
void SLTPopBack(SListNode** pphead) {
    assert(pphead);
    assert(*pphead != NULL);
    if ((*pphead)->next == NULL) {
        free(*pphead);
        *pphead = NULL;
    } else {
        SListNode* prev = NULL;
        SListNode* tail = *pphead;
        while (tail->next != NULL) {
            prev = tail;
            tail = tail->next;
        }
```

```
        free(tail);
        prev->next = NULL;
    }
}
```

（6）遍历链表

Code block

```
// 打印单链表
void SListPrint(SListNode* phead) {
    SListNode* cur = phead;
    while (cur != NULL) {
        printf("%d -> ", cur->data);
        cur = cur->next;
    }
    printf("NULL\n");
}
```

（7）反转链表

Code block

```
// 反转单链表
void SListReverse(SListNode** pphead) {
    assert(pphead);
    if (*pphead == NULL || (*pphead)->next == NULL) {
        return;
    }
    SListNode* prev = NULL;
    SListNode* cur = *pphead;
    SListNode* next = NULL;
    while (cur != NULL) {
        next = cur->next;
        cur->next = prev;
        prev = cur;
        cur = next;
    }
    *pphead = prev;
}
```

（8）销毁链表

Code block

```c
// 销毁单链表
void SListDestroy(SListNode** pphead) {
    assert(pphead);
    SListNode* cur = *pphead;
    while (cur != NULL) {
        SListNode* next = cur->next;
        free(cur);
        cur = next;
    }
    *pphead = NULL;
}
```

3.2 单链表完整示例

Code block

```c
int main() {
    SListNode* plist = NULL;
    // 尾插节点
    SLTPushBack(&plist, 1);
    SLTPushBack(&plist, 2);
    SLTPushBack(&plist, 3);
    SListPrint(plist); // 1 -> 2 -> 3 -> NULL

    // 头插节点
    SLTPushFront(&plist, 0);
    SListPrint(plist); // 0 -> 1 -> 2 -> 3 -> NULL

    // 删除头节点
    SLTPopFront(&plist);
    SListPrint(plist); // 1 -> 2 -> 3 -> NULL

    // 删除尾节点
    SLTPopBack(&plist);
    SListPrint(plist); // 1 -> 2 -> NULL

    // 反转链表
    SListReverse(&plist);
    SListPrint(plist); // 2 -> 1 -> NULL

    // 销毁链表
    SListDestroy(&plist);
    SListPrint(plist); // NULL

    return 0;
}
```

四、双向链表的实现与操作

4.1 基本操作

（1）创建节点

Code block

```
// 创建双向链表新节点
DListNode* BuyDListNode(int x) {
    DListNode* newnode = (DListNode*)malloc(sizeof(DListNode));
    if (newnode == NULL) {
        perror("malloc failed");
        exit(-1);
    }
    newnode->data = x;
    newnode->prev = NULL;
    newnode->next = NULL;
    return newnode;
}
```

（2）初始化带头双向循环链表

Code block

```
// 初始化带头双向循环链表
DListNode* DListInit() {
    DListNode* phead = BuyDListNode(0); // 头节点数据域无意义
    phead->next = phead;
    phead->prev = phead;
    return phead;
}
```

（3）在指定位置前插入节点

Code block

```
// 在双向循环链表的pos位置之前插入新节点
void DListInsert(DListNode* pos, int x) {
    assert(pos);
    DListNode* newnode = BuyDListNode(x);
    DListNode* prev = pos->prev;
    // 连接节点
    prev->next = newnode;
```

```
    newnode->prev = prev;
    newnode->next = pos;
    pos->prev = newnode;
}
```

（4）删除指定节点

Code block

```
// 删除双向循环链表中指定位置pos的节点
void DListErase(DListNode* pos) {
    assert(pos);
    DListNode* prev = pos->prev;
    DListNode* next = pos->next;
    prev->next = next;
    next->prev = prev;
    free(pos);
}
```

（5）遍历链表

Code block

```
// 打印带头双向循环链表
void DListPrint(DListNode* phead) {
    assert(phead);
    DListNode* cur = phead->next;
    printf("NULL <- ");
    while (cur != phead) {
        printf("%d", cur->data);
        if (cur->next != phead) {
            printf(" <-> ");
        }
        cur = cur->next;
    }
    printf(" -> NULL\n");
}
```

（6）销毁链表

Code block

```
// 销毁带头双向循环链表
void DListDestroy(DListNode** pphead) {
    assert(pphead && *pphead);
```

```
    DListNode* cur = (*pphead)->next;
    while (cur != *pphead) {
        DListNode* next = cur->next;
        free(cur);
        cur = next;
    }
    free(*pphead);
    *pphead = NULL;
}
```

4.2 双向链表完整示例

Code block

```
int main() {
    DListNode* plist = DListInit();
    // 插入节点
    DListInsert(plist->next, 10);
    DListInsert(plist->next, 20);
    DListInsert(plist->prev, 30);
    DListPrint(plist); // NULL <- 20 <-> 10 <-> 30 -> NULL

    // 删除节点
    DListErase(plist->next->next);
    DListPrint(plist); // NULL <- 20 <-> 30 -> NULL

    // 销毁链表
    DListDestroy(&plist);
    return 0;
}
```

五、循环链表的实现与操作

5.1 单向循环链表基本操作

（1）创建节点

Code block

```
// 创建循环链表节点
CListNode* BuyCListNode(int x) {
    CListNode* newnode = (CListNode*)malloc(sizeof(CListNode));
    if (newnode == NULL) {
        perror("malloc failed");
        exit(-1);
```

```
    }
    newnode->data = x;
    newnode->next = newnode; // 初始指向自身
    return newnode;
}
```

（2）尾插操作

Code block

```
// 在单向循环链表尾部插入节点
void CListPushBack(CListNode** pphead, int x) {
    CListNode* newnode = BuyCListNode(x);
    if (*pphead == NULL) {
        *pphead = newnode;
    } else {
        CListNode* tail = *pphead;
        while (tail->next != *pphead) {
            tail = tail->next;
        }
        tail->next = newnode;
        newnode->next = *pphead;
    }
}
```

（3）遍历链表

Code block

```
// 打印单向循环链表
void CListPrint(CListNode* phead) {
    if (phead == NULL) {
        printf("链表为空\n");
        return;
    }
    CListNode* cur = phead;
    do {
        printf("%d -> ", cur->data);
        cur = cur->next;
    } while (cur != phead);
    printf("(HEAD)\n");
}
```

（4）销毁链表

Code block

```
// 销毁单向循环链表
void CListDestroy(CListNode** pphead) {
    if (*pphead == NULL) {
        return;
    }
    CListNode* cur = (*pphead)->next;
    while (cur != *pphead) {
        CListNode* next = cur->next;
        free(cur);
        cur = next;
    }
    free(*pphead);
    *pphead = NULL;
}
```

5.2 单向循环链表完整示例

Code block

```
int main() {
    CListNode* plist = NULL;
    CListPushBack(&plist, 1);
    CListPushBack(&plist, 2);
    CListPushBack(&plist, 3);
    CListPrint(plist); // 1 -> 2 -> 3 -> (HEAD)

    CListDestroy(&plist);
    return 0;
}
```

六、链表的高级技巧

6.1 快慢指针技巧

（1）检测链表是否有环

Code block

```
// 检测单链表是否有环
int hasCycle(SListNode* head) {
    SListNode* slow = head;
    SListNode* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
```

```
        if (slow == fast) {
            return 1; // 有环
        }
    }
    return 0; // 无环
}
```

（2）找到链表的中间节点

Code block

```
// 查找单链表的中间节点
SListNode* findMiddle(SListNode* head) {
    SListNode* slow = head;
    SListNode* fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow; // 慢指针指向中间节点
}
```

6.2 链表排序

归并排序链表

Code block

```
// 合并两个有序单链表
SListNode* mergeTwoLists(SListNode* l1, SListNode* l2) {
    if (l1 == NULL) return l2;
    if (l2 == NULL) return l1;
    SListNode* result = NULL;
    if (l1->data <= l2->data) {
        result = l1;
        result->next = mergeTwoLists(l1->next, l2);
    } else {
        result = l2;
        result->next = mergeTwoLists(l1, l2->next);
    }
    return result;
}

// 归并排序单链表
SListNode* mergeSort(SListNode* head) {
    if (head == NULL || head->next == NULL) {
```

```
        return head;
    }
    SListNode* middle = findMiddle(head);
    SListNode* nextOfMiddle = middle->next;
    middle->next = NULL;
    SListNode* left = mergeSort(head);
    SListNode* right = mergeSort(nextOfMiddle);
    return mergeTwoLists(left, right);
}
```

6.3 删除链表的倒数第 N 个节点

Code block

```
// 删除单链表的倒数第N个节点
SListNode* removeNthFromEnd(SListNode* head, int n) {
    SListNode* dummy = BuySListNode(0);
    dummy->next = head;
    SListNode* fast = dummy;
    SListNode* slow = dummy;
    // 快指针先移动n+1步
    for (int i = 0; i <= n; i++) {
        if (fast == NULL) {
            return head; // n超出链表长度
        }
        fast = fast->next;
    }
    // 快慢指针同时移动
    while (fast != NULL) {
        slow = slow->next;
        fast = fast->next;
    }
    // 删除节点
    SListNode* temp = slow->next;
    slow->next = slow->next->next;
    free(temp);
    return dummy->next;
}
```

七、链表的应用场景

1. **动态数据存储**：当数据量不确定或频繁变化时，链表可以提供更灵活的存储方式。
2. **实现栈和队列**：链表可以轻松实现栈的 “后进先出” （LIFO）和队列的 “先进先出” （FIFO）特性。

3. **操作系统内存管理**：操作系统中常使用链表来管理空闲内存块。
4. **图的邻接表表示**：在图结构中，邻接表通常使用链表来存储每个顶点的邻接点。
5. **LRU 缓存淘汰算法**：链表可以用于实现最近最少使用的缓存淘汰策略。
6. **文本编辑器的撤销栈**：利用链表的特性实现撤销操作的历史记录。
7. **播放器的播放列表**：支持动态添加、删除和调整播放顺序。

八、注意事项

8.1 内存管理

- 记得释放不再使用的节点内存，避免内存泄漏。
- 使用 `malloc` 分配内存后，要检查是否分配成功，避免空指针访问。
- 销毁链表时要遍历所有节点，逐个释放内存，不能只释放头节点。

8.2 边界处理

- 注意处理空链表、单节点链表等特殊情况。
- 在插入和删除节点时，要确保指针的指向正确，避免出现野指针。
- 双向链表和循环链表的操作要注意指针的双向维护，避免断链。

8.3 指针操作

- 小心指针的指向关系，避免出现野指针和空指针引用。
- 在修改指针指向时，要先保存必要的指针，避免丢失节点的引用。
- 带头链表可以简化边界条件的处理，建议在工程实践中使用带头链表。

（注：文档部分内容可能由 AI 生成）